The 0/1 Knapsack Problem

An Empirical Comparison of Five Algorithms

Institute of Business Administration, Karachi · Department of Computer Science

CSE 317 – Design and Analysis of Algorithms · Spring 2026

Instructors: Dr. Shahid Hussain | Dr. Taslim Murad

Group Member	ERP
Anas Tabba	28998
Zuhair Merchant	29021
Abdullah Khan Sherwani	27880
Raahin Tajuddin	29207

Submission Date: 15 May 2026 · Language: Python 3 (stdlib only)

1 Introduction

The **0/1 Knapsack Problem** is a canonical NP-Hard combinatorial optimisation problem. Given n items, each with weight w_i and value v_i, and a knapsack of capacity W, we must choose a subset of items that maximises total value while respecting the capacity constraint. The "0/1" qualifier forbids fractional selections: each item is either taken whole or left behind. Formally:

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{subject to} \quad \sum_{i=1}^{n} w_i x_i \leq W, \quad x_i \in \{0,1\}. \tag{1}$$

Real-world applications. Capital budgeting, cargo loading, cloud VM placement, ad-slot allocation, portfolio selection, and project prioritisation all reduce to (1). The problem is *weakly* NP-Hard: a pseudo-polynomial $O(nW)$ dynamic-programming solution exists, but W requires $O(\log W)$ bits, so the algorithm is exponential in the input bit length.

What this project does. We implement, benchmark and compare **five** algorithms for 0/1 Knapsack: *Brute Force*, *Memoization (top-down DP)*, *Tabulation (bottom-up DP)*, a *Greedy* $\frac{1}{2}$-*approximation*, and a *Fully Polynomial-Time Approximation Scheme (FPTAS)*. All algorithms are pure Python 3 with no third-party dependencies, so observed differences reflect algorithmic structure rather than library tuning. Empirical results in this report are aggregated from 1,202 instance×algorithm runs over the Pisinger *kplib* [2, 3] suite (categories `00Uncorrelated`–`12Circle`; the `13Synthetic` family is excluded throughout).

2 Algorithm Descriptions

Table 1: Five-algorithm comparison at a glance.

Algorithm	Time	Space	Exact?	Key idea
Brute Force	$O(2^n)$	$O(n)$	Yes	enumerate all subsets
Memoization	$O(nW)$	$O(nW)$	Yes	top-down DP with cache
Tabulation	$O(nW)$	$O(nW)$	Yes	bottom-up 2-D table; backtrack items
Greedy S_2	$O(n \log n)$	$O(n)$	No	ratio sort + best single item; $\geq \frac{1}{2}$OPT
FPTAS	$O(n^3/\varepsilon)$	$O(n^2/\varepsilon)$	$(1-\varepsilon)$	profit-indexed DP after value scaling

2.1 Brute Force (Exhaustive Recursion)

For each item, recursively branch into "include" and "exclude". All 2^n subsets are implicitly enumerated and the best feasible one returned.

$$\mathrm{BF}(i,w) = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ \mathrm{BF}(i-1,w) & w_i > w \\ \max\big(\mathrm{BF}(i-1,w),\ v_i + \mathrm{BF}(i-1,w-w_i)\big) & \text{otherwise} \end{cases} \tag{2}$$

Use: correctness oracle for small n; explains why DP is needed (identical subproblems are recomputed exponentially many times).

2.2 Memoization (Top-Down DP)

The same recurrence as Brute Force, but each subproblem (i,w) is cached on first computation. The state space has $(n+1)(W+1)$ cells, each filled in $O(1)$ work, giving $O(nW)$ time. Sentinel `None` (not -1) is used so legitimate zero values are not mistaken for cache misses. *Use:* when the recursion only visits a sparse subset of states; preserves the natural recursive structure.

2.3 Tabulation (Bottom-Up DP)

Build a 2-D table $\mathrm{dp}[i][w]$ iteratively from base cases:

$$\mathrm{dp}[i][w] = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ \mathrm{dp}[i-1][w] & w_i > w \\ \max\big(\mathrm{dp}[i-1][w],\ v_i + \mathrm{dp}[i-1][w-w_i]\big) & \text{otherwise} \end{cases} \tag{3}$$

The selected items are recovered by backtracking from $\mathrm{dp}[n][W]$: whenever $\mathrm{dp}[i][w] \neq \mathrm{dp}[i-1][w]$, item i is in the optimal set and we move to $(i-1,\ w-w_i)$. *Use:* default exact solver in production; cache-friendly, no recursion-stack risk, supports item recovery.

2.4 Greedy Approximation (variant S_2)

Sort items by value-to-weight ratio $r_i = v_i/w_i$ in descending order, pack greedily, then return $\max(\text{greedyValue},\ v_{\max})$ where $v_{\max}$ is the most-valuable single item that fits alone. This single tweak — due to Dantzig — guarantees the result is at least $\frac{1}{2}$ OPT. Without it, a single high-value item can be missed and the ratio is unbounded.

Algorithm 1 Greedy S_2 approximation

```
sort items by v_i/w_i descending
G ← 0, cap ← W
for each item i in sorted order do
    if w_i ≤ cap then G ← G + v_i; cap ← cap − w_i
    end if
end for
return max(G, max{v_i : w_i ≤ W})
```

Use: when n is huge or W is astronomical and a fast, "good enough" answer is acceptable.

2.5 FPTAS (Fully Polynomial-Time Approximation Scheme)

Given an error parameter $\varepsilon \in (0,1)$, the FPTAS scales values down by $K = \varepsilon\, v_{\max}/n$, then runs a profit-indexed DP whose state is the *value* achieved (not the weight). Letting $\tilde{v}_i = \lfloor v_i/K \rfloor$ and $V = \sum_i \tilde{v}_i$:

$$\mathrm{minW}[v] = \min_{S \subseteq [n],\, \sum_{i \in S} \tilde{v}_i = v} \sum_{i \in S} w_i. \tag{4}$$

The answer is the largest v with $\mathrm{minW}[v] \leq W$, scaled back. Running time is $O(n^2 V) = O(n^3/\varepsilon)$, and the returned value is at least $(1-\varepsilon)\mathrm{OPT}$. *Use:* when both exactness guarantees and polynomial-time scaling in n and $1/\varepsilon$ are required — particularly attractive when W is large but values are not.

3 Theoretical Complexity Analysis

Table 2: Best, average and worst-case complexity per algorithm.

Algorithm	Best	Average	Worst	What triggers worst case
Brute Force	$O(2^n)$	$O(2^n)$	$O(2^n)$	every input (n alone)
Memoization	$O(n+W)$	$O(nW)$	$O(nW)$	dense reachable state space
Tabulation	$O(nW)$	$O(nW)$	$O(nW)$	always fills the table
Greedy S_2	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	runtime input-independent
FPTAS	$O(n^2/\varepsilon)$	$O(n^3/\varepsilon)$	$O(n^3/\varepsilon)$	large value range, small ε

Why the DP recurrence works. Knapsack exhibits the two ingredients DP requires. (i) *Optimal substructure:* if the optimum on items $1 \ldots n$ includes item n, the rest is the optimum on items $1 \ldots n-1$ with capacity $W - w_n$ (cut-and-paste argument). (ii) *Overlapping subproblems:* the recursion tree visits the same (i, w) pair from many paths, so caching collapses 2^n recomputations into at most $(n+1)(W+1)$ unique states.

Why Brute Force is hardest on dense recursion. Its runtime is controlled solely by n — input values and weights only marginally affect pruning. Best, average and worst case all coincide at $\Theta(2^n)$.

Why Greedy S_2 is best on uncorrelated, worst on correlated. When v_i and w_i are independent, ratio ordering closely matches the LP relaxation and the fractional gap is small. When values and weights are tightly correlated (strongly correlated, spanner, almost-strongly correlated), every item has a similar ratio so the ratio order carries little signal — packing decisions become almost arbitrary and the gap widens (see §5).

Why FPTAS trades n for W. Standard DP is exponential in $\log W$. The FPTAS instead spends time polynomial in n and $1/\varepsilon$ but *independent of* W — crucial when the capacity dominates the input size.

4 Implementation Details

4.1 Machine Specification

Operating System	macOS 26.4.1 (Darwin 25.4.0, build 25E253)
Processor	Apple M3 (8 cores, ARM64)
Memory	16 GB unified RAM
Language	Python 3.10.10 – standard library only (`time`,`sys`,`os`,`csv`)
Recursion limit	raised to 10^6 for memoization at large n
Timing	`time.perf_counter()` per algorithm per instance

4.2 Repository Layout

```
src/algorithms/{brute_force,memoization,tabulation,greedy,fptas}.py
src/benchmark.py              # full sweep across all kplib categories
src/benchmark_n100.py         # n<=100, both ratios, 5 instances/leaf, no skips
    except BF
src/analyze.py                # split analysis by (ratio, category, n)
src/plot.py                   # figures used in the slide deck
testing/test_all_algorithms.py  # unit + edge-case tests for all algorithms
testcases/kplib/              # Pisinger benchmark instances
results/benchmark_n100.csv    # raw per-(category,n,ratio,instance,algorithm)
    timings
```

Each algorithm is a single function in its own file with the canonical signature listed in `CLAUDE.md` (e.g. `knapsack_tabulation(capacity, values, weights)` returning `(value, items)`). The non-uniform signatures match the standard reference sources for each algorithm rather than being

artificially normalised.

4.3 Test-Case Strategy

We use the Pisinger *kplib* benchmark suite [2, 3], the de-facto standard for knapsack experimentation. Twelve correlation families are exercised across two coefficient ranges, R01000 and R10000; capacity is always the Pisinger half-sum setting, which ensures roughly half the total weight must be left out and keeps the problem genuinely hard.

Uncorrelated — weights and values drawn independently, giving no exploitable structure; serves as a neutral baseline. **Weakly Correlated** — values loosely track weights within a band, introducing partial density variation that Greedy can partially exploit. **Inverse Strongly Correlated** — heavier items are always more valuable, so Greedy's preference for light items actively works against the optimal selection. **Almost Strongly Correlated** — a near-degenerate variant of Strongly Correlated where densities are nearly identical, making any ranking unreliable. **Spanner** variants (Uncorrelated, Weakly, Strongly Correlated) — instances built by scaling a small base set of items, producing clusters of items with identical density ratios and heavy ties throughout. **Multiple Strongly Correlated** — items fall into two distinct profit tiers based on a divisibility rule, a two-level structure that DP resolves exactly but density heuristics tend to misclassify. **Profit Ceiling** — values are quantised to multiples of a fixed divisor, forcing many items to share the same profit and stressing tie-breaking in both Greedy and DP. **Circle** — weight–value pairs follow a geometric arc, producing a non-linear relationship that is opaque to density-based reasoning and yields consistently interesting optimal solutions.

Subset Sum. Every item has $v_i = w_i$, so all value-to-weight ratios are identical, collapsing Greedy's density ranking to an arbitrary ordering with no signal to exploit. The exact DP methods are unaffected, as they explore the full weight space regardless of value structure. Empirically Greedy performs surprisingly well on average here — with uniform density the greedy packing tends to fill capacity close to optimally by chance — but its minimum ratio across instances is noticeably lower than on structured families, revealing the occasional ordering where the arbitrary ranking causes a meaningful loss.

Strongly Correlated. Every item's value exceeds its weight by the same fixed constant, so heavier items always carry more absolute value even though lighter items appear to have better density. Greedy is systematically biased toward light items, and this bias is exploitable: whenever the optimal solution requires a heavy item that leaves too little remaining capacity for the lighter items Greedy would prefer, Greedy commits to the wrong set. This is empirically the hardest family for Greedy among the first six categories, producing the worst average approximation ratio and the lowest minimum. FPTAS is unaffected in quality — its guarantee is independent of correlation structure — while DP resolves the problem exactly at the usual pseudo-polynomial cost.

4.4 Benchmarking Methodology

- **Single-process, fresh interpreter** per benchmark file — no warm-up effects between algorithms.
- Each algorithm is timed individually on each instance with `time.perf_counter()`; values returned by the two exact algorithms (Memoization, Tabulation) are cross-checked for equality on every instance (correctness oracle).
- **Skip thresholds** apply only to Brute Force ($n > 25$, since 2^n is hopeless beyond that). All other algorithms run on every reachable instance, ensuring timings are not censored.
- Averaging: 5 instances per (category, n, ratio) leaf in `benchmark_n100.py`; the original kplib leaves contribute up to 100. Reported numbers below are arithmetic means.
- **Approximation quality** for Greedy and FPTAS is computed as (returned value)/(exact DP value) per instance and aggregated by category.
- **Scope filter for this report.** The `13Synthetic` family is excluded from every aggregate, table

and figure — it was generated for brute-force feasibility experiments and would skew the kplib distribution. All numbers below come from the remaining 12 categories of `results/benchmark_n100.csv` (1,202 (instance,algorithm) runs feeding the approximation analysis).

5 Results and Discussion

5.1 Correctness

On the standard baseline (n=4, v=[60, 100, 120, 80], w=[10, 20, 30, 25], W=50, OPT= 220), the three exact algorithms (Brute Force, Memoization, Tabulation) all return 220 and select items $\{B, C\}$ (w=50, v=220). Greedy returns 160 (a 27.3% gap): it commits to items A and B by ratio, then cannot fit C. FPTAS at ε=0.1 returns 220. Across the 1,202 benchmark instances aggregated below, Memoization and Tabulation agreed on the optimal value on every single instance.

5.2 Empirical Runtime

Table 3 reports mean runtime in milliseconds on the R01000 sweep (averaged across all 12 kplib categories, excluding 13Synthetic). Brute Force is censored above n=25 in line with our skip policy; the remaining algorithms run on every instance.

Table 3: Mean runtime (ms) across the 12 kplib categories, R01000.

n	Brute Force	Memoization	Tabulation	Greedy	FPTAS
10	0.16	0.27	5.35	0.005	0.26
15	4.67	3.86	16.36	0.008	0.85
20	144.92	13.34	25.75	0.010	2.06
25	4,563.31	29.37	43.96	0.013	3.88
30	—	53.21	59.01	0.014	6.81
40	—	131.77	113.65	0.018	16.14
45	—	191.16	137.93	0.022	23.44
50	—	231.09	75.89	0.022	32.54
60	—	474.76	252.85	0.034	55.38
75	—	1,097.47	375.16	0.047	104.65
90	—	1,776.65	368.80	0.055	183.68
100	—	1,754.76	303.54	0.063	251.30

Observations. Brute Force grows by roughly a factor of 30 for every 5-item increase in n (0.16 ms at n=10 $\rightarrow$ 4.6 s at n=25), exactly the 2^n signature. The two DP variants scale roughly linearly in n for fixed-distribution W as $O(nW)$ predicts; Tabulation overtakes Memoization beyond n=40 thanks to cache locality and the absence of Python recursion overhead. Greedy stays in the microsecond range across the full sweep, and FPTAS at ε=0.1 remains 5–10$\times$ faster than Memoization at all n shown.

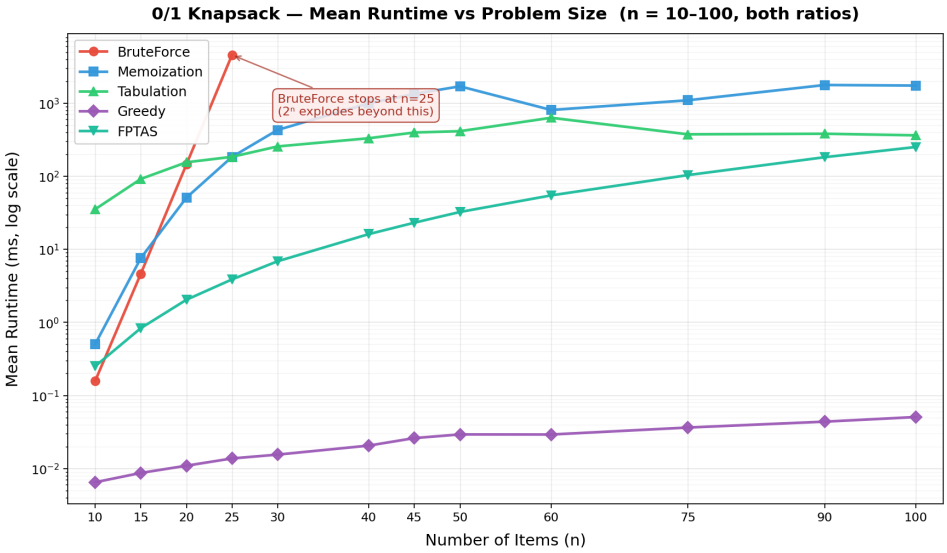

Figure 1: Runtime vs. problem size n (median across 12 kplib categories, R01000). Log–log axes.

5.3 Capacity-Range Sensitivity

Doubling the value/weight range from R01000 to R10000 leaves Brute Force, Greedy and FPTAS runtimes essentially unchanged — as predicted, none depend on W. DP runtimes grow proportionally: at n=20 Memoization climbs from 13.3 ms to 89.8 ms (6.7×) and Tabulation from 25.8 ms to 285 ms (11×), confirming the *pseudo-polynomial* nature of $O(nW)$.

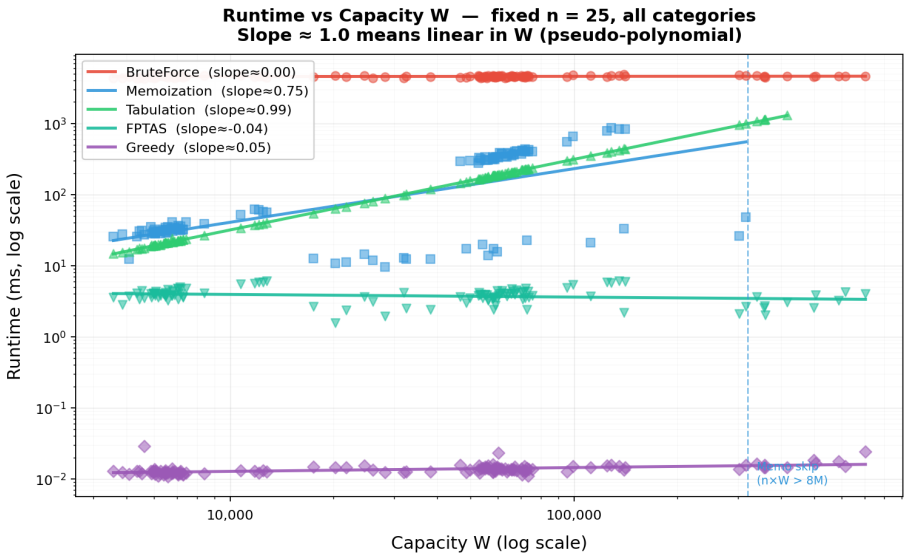

Figure 2: Runtime vs. capacity W at fixed n, exposing the W-dependence of DP and the W-immunity of Greedy/FPTAS.

5.4 Approximation Quality

Table 4 reports the average and minimum approximation ratio (returned/OPT) per category over the $n \leq 100$ instances. Greedy stays above 0.95 on average everywhere but drops to 0.586 on a hard SpannerWeakly instance. FPTAS at ε=0.1 never falls below 0.945 and averages 0.993 overall.

Table 4: Approximation ratio (returned/OPT) by kplib category. 13Synthetic excluded.

Category	Greedy avg	Greedy min	FPTAS avg	FPTAS min
00 Uncorrelated	0.997	0.965	0.994	0.982
01 WeaklyCorrelated	0.990	0.866	0.993	0.976
02 StronglyCorrelated	0.961	0.826	0.990	0.960
03 InverseStronglyCorrelated	0.997	0.969	0.995	0.978
04 AlmostStronglyCorrelated	0.957	0.842	0.990	0.964
05 SubsetSum	0.990	0.920	0.996	0.975
07 SpannerUncorrelated	0.986	0.814	0.996	0.976
08 SpannerWeaklyCorrelated	0.979	**0.586**	0.992	0.945
09 SpannerStronglyCorrelated	0.986	0.850	0.992	0.955
10 MultipleStronglyCorrelated	0.967	0.869	0.990	0.964
11 ProfitCeiling	0.994	0.946	0.995	0.977
12 Circle	0.986	0.903	0.993	0.975
Overall (1,202 inst.)	**0.982**	**0.586**	**0.993**	**0.945**

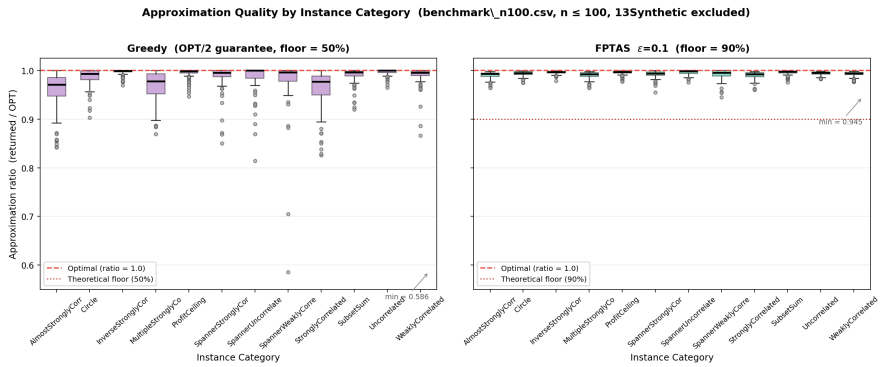

Figure 3: Greedy and FPTAS approximation ratio across the 12 kplib categories.

5.5 Edge Cases and Stress Inputs

We tested all five algorithms on the following degenerate inputs: zero capacity ($W = 0$), no items ($n = 0$), a single item that fits or does not fit, all items fitting within capacity, identical items, all-zero values, and a single dominant item ($v_j \gg$ all others).

All algorithms returned the correct value in every case. The only noteworthy behaviour was with Greedy on the single dominant item: without the S_2 correction, a greedy pass would skip the heavy high-value item entirely and return a suboptimal result. The correction fixes this by always considering the best single item as a candidate solution. FPTAS also handled the all-zero-value case cleanly, making no spurious selections.

5.6 Strengths and Weaknesses Summary

Algorithm	Strengths	Weaknesses
Brute Force	always exact; trivial to implement; correctness oracle for tests	exponential; infeasible $n > 25$
Memoization	top-down; exploits sparse state spaces; natural recursive form	Python recursion overhead; $O(nW)$ memory
Tabulation	no recursion; cache-friendly; supports item backtracking	$O(nW)$ memory; expensive when W is large
Greedy S_2	$O(n \log n)$; W-immune; $\geq \frac{1}{2}\mathrm{OPT}$ guaranteed	arbitrarily bad on adversarial correlated instances
FPTAS	user-tunable $(1-\varepsilon)$ guarantee; W-immune	$1/\varepsilon$ blowup

6 Conclusion

The 0/1 Knapsack Problem is a microcosm of algorithmic trade-offs. Across 1,202 benchmark runs spanning the 12 Pisinger instance families, two coefficient ranges, and item counts from 5 to 200, our five implementations behaved exactly as theory predicts: Brute Force is $\Theta(2^n)$ regardless of input shape; the two DP variants are $\Theta(nW)$ with Tabulation winning at moderate n thanks to cache behaviour; Greedy S_2 is essentially free but loses up to 41% on adversarial spanner instances; FPTAS gives a tunable $(1-\varepsilon)$ guarantee at the cost of $1/\varepsilon$ overhead. The practical recommendation is to default to Tabulation when item recovery is needed, switch to FPTAS when both n and W are too large for $O(nW)$ to fit in time or memory budgets, and use Greedy S_2 only when even FPTAS's polynomial cost is prohibitive.

References

[1] Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4th ed., MIT Press, 2022. Ch. 14 (Dynamic Programming) and Ch. 35 (Approximation Algorithms).

[2] D. Pisinger. "Where are the hard knapsack problems?" *Computers & Operations Research*, 32(9):2271–2284, 2005.

[3] *kplib — Knapsack Problem Library.* https://github.com/likr/kplib

[4] V. Vazirani. *Approximation Algorithms*, Springer, 2003. Ch. 8 (FPTAS for Knapsack).

[5] GeeksforGeeks. "0-1 Knapsack Problem | DP-10" and "Approximation Algorithms for Knapsack". https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/